ECE M214A: Digital Speech Processing
Noise Robust Digit Recognition

Department of Electrical and Computer Engineering
UCLA
Winter 2026

Due Dates

Oral presentations and code submission are scheduled for Wednesday March 11th during class time (10 am - 12 pm). For MSOL students, it is scheduled during the final exam slot, Saturday March 14th 1-3 pm, by Zoom. Project reports are due on March 16th at 10 a.m.

1 Introduction

In this project, we are interested in implementing a robust spoken digit recognition system that accurately identifies spoken digits (0-9) from audio recordings. Acoustic features such as MFCCs, LPCs (and others) may be used to perform the task. The challenges of this task arise both due to the limited training data, and the mismatch between the training and test conditions when noise is present.

2 Data

The Free Spoken Digit Dataset (FSDD) [1] contains recordings of spoken digits (0-9) from 6 different speakers. The dataset consists of 3,000 recordings total, with 50 recordings of each digit per speaker. Files are named in the format `{digitLabel}_{speakerName}_{index}.wav` (e.g., `7_jackson_32.wav`). We use all recordings from the FSDD dataset, which have been trimmed to contain minimal silence at the beginnings and ends of the utterances.

Your model will be also tested on a blind test set. The blind test set will consist of a different set of noisy utterances from the same speakers. All the wav files provided to you were originally sampled at 8kHz.

3 Project Codebase

The enclosed project link provides access to the following folders with utterances: `train_clean`, `test_clean`, `test_noisy_5dB`, `test_noisy_10dB`. That is, the test utterances are in one dataset "clean" and in the other, a noise masker (additive) was added (babble noise added at SNRs of 5 dB and 10 dB). The following python notebook [2] contains functions that extract features from the utterances in the above folders, and calculates the accuracy through the use of a LSTM based classifier.

Details: The baseline system consists of the following: Frame-wise MFCC features were extracted from each utterance at a frame rate of 100 Hz ('n_mfcc=13' and 'win_size=25ms') and used to train the LSTM model. More detailed information and instructions on the specific libraries used for feature extraction and the LSTM model are present within the python notebook.

4 Objectives

Your task is to derive a set of features and implement them to predict the spoken digit (0-9) for every utterance. You will train on clean data and test with 1) clean data, and 2) noisy data at SNRs of 5 dB and 10 dB. You can run the script either on google colab, or on a personal laptop with GPU.

5 Evaluation Metrics

Evaluation of the project will be primarily based on the performance of the trained classifier. The accuracy calculated by the Python script will be used for evaluation. In addition to accuracy, report other classification metrics such as the confusion matrix of your model. Results on both clean and noisy test sets, especially noisy test set, as well as performance on the blind test set, will be considered in the evaluation.

Important

When reporting your main results, do not modify the baseline LSTM classifier (including the learning rate or batch size), switch to a different classifier, or train on noisy data. The primary objective of this project is to develop a robust feature set. Exploring alternative classifiers and data augmentation techniques is encouraged, but should be treated as bonus work and reported separately.

Bonus (Optional)

You are encouraged to explore one or both of the following extensions, though they will contribute minimally to your overall score:

1. Experiment with different classifiers such as MLP, CNN, Transformer etc.
2. Implement data augmentation techniques to improve noise robustness

6 Instructions

6.1 Setting up the project

a. Download project data from Google Drive [3].

b. Upload the 'M214_project_data.zip' file to your Google Drive.

c. Make a copy of the colab notebook [2].

d. Open the notebook, modify path to your zip file and run all the cells.

6.2 Run custom features from Python

a. In the accompanying python notebook, edit the `extract_features` function to calculate your custom features directly.

b. Run the model training and inference steps directly.

c. Repeat for other trials.

7 Baseline Results

The baseline script using MFCCs should take around 5 mins to run on Google Colab using a GPU runtime. You may need to change the seed and run several times to get the best results.

Dataset	**Accuracy**
Test Clean	97%
Test Noisy 10dB	70%
Test Noisy 5dB	51%

8 Oral Presentations

There will be oral presentations by the different teams describing their work. Presentations should be planned by the team as a group.

9 Report and Code

The report (one per group) should include:

- Introduction (what is the problem/why is it important)
- Background (literature survey on digit recognition and noise robustness)
- Project Description (features, algorithm, implementation, results, average run times, etc.)
- Summary and Discussion (also ideas for future work)
- References (cited throughout the report)
- Figures and flowcharts generally help clarify the text.

The report should be 4-pages long (excluding references) and have the same format as the INTERSPEECH conference [7].

The code should be turned in on the day of the presentation. Comments at the beginning of each function should describe what the function intends to do.

References and supporting materials

[1] Free Spoken Digit Dataset (FSDD). https://github.com/Jakobovski/free-spoken-digit-dataset

[2] Project Colab Notebook: https://colab.research.google.com/drive/1lfIy0O3paEq1e0uvAlwcfQD2KA1TMPy9?usp=sharing

[3] Project Data Package:https://drive.google.com/file/d/1jZ0MWj7mzKWWY0zSP_fgaiRbXrLb3FcN/view?usp=sharing

[4] OpenSMILE audio feature sets: https://github.com/audeering/opensmile-python

[5] Noise robust features: Kim C, Stern R M. Power-normalized cepstral coefficients (PNCC) for robust speech recognition. IEEE/ACM Transactions on Audio, Speech and Language Processing (TASLP), 2016, 24(7): 1315-1329.

[6] Useful toolkit for speech processing and feature extraction: https://librosa.org/

[7] Interspeech Template: https://interspeech2026.org/en-AU/pages/author-resources/resources